

TUGAS PRAKTIKUM 2

Mata Kuliah: Pemrograman Basis Data *dengan C#*

Topik: Penggunaan Component Controls & List Controls

Mata Kuliah: Pemrograman Basis Data

Pengajar: Drs. Bambang Sujatmiko, M.T.

Materi: Translasi Model Konseptual (*Conceptual Data Model*) ke Model Fisik (*Physical Data Model*)

Materi:Praktik Kueri Relasional Multi-Tabel pada Ekosistem C# .NET dan Microsoft SQL Server

MODUL AJAR DAN PANDUAN PRAKTIKUM

Mata Kuliah: Pemrograman Basis Data

Bobot: 3 SKS (1 SKS Teori, 2 SKS Praktikum)

Topik Pembelajaran: Translasi Model Konseptual (*Conceptual Data Model*) ke Model Fisik (*Physical Data Model*) dan Kueri Relasional Multi-Tabel pada Ekosistem C# .NET dan Microsoft SQL Server

Target Format Dokumen: Salinan Terstruktur Siap Konversi ke .docx

BAGIAN I: MODUL AJAR (LANDASAN TEORI & MODEL RELASIONAL)

1. Capaian Pembelajaran dan Indikator Kinerja

Pengembangan sistem perangkat lunak berorientasi data menuntut pemahaman mendalam mengenai bagaimana representasi logis dari domain bisnis dialihragamkan menjadi struktur tabel fisik yang efisien pada sistem manajemen basis data relasional (*Relational Database Management System / RDBMS*). Modul ajar ini disusun berdasarkan prinsip *Outcome-Based Education* (OBE) untuk menjembatani perancangan diagram hubungan entitas (*Entity-Relationship Diagram / ERD*) dengan manipulasi data multi-tabel menggunakan Transact-SQL (T-SQL) serta pemrogramannya pada platform desktop C# Visual Studio.

Tabel 1. CP dan CPMK

Komponen Kurikulum	Deskripsi Capaian Pembelajaran	Bobot Kinerja
Capaian Pembelajaran Mata Kuliah (CPMK)	Mahasiswa mampu mentranslasikan rancangan model data konseptual menjadi model data fisik yang teroptimasi, serta mengekstraksi informasi relasional multi-tabel secara aman melalui aplikasi desktop berbasis GUI.	100%
Sub-CPMK 1 (Translasi PDM)	Mahasiswa mampu memetakan entitas, atribut, dan relasi ERD ke dalam skema tabel fisik, tipe data, serta batasan integritas referensial (<i>integrity constraints</i>) di SQL Server.	25%

Komponen Kurikulum	Deskripsi Capaian Pembelajaran	Bobot Kinerja
Sub-CPMK 2 (Aljabar Relasional & Join)	Mahasiswa mampu membedakan karakteristik himpunan hasil dari operasi penggabungan tabel (<i>Inner, Left, Right, Full Outer</i>) serta menganalisis dampak anomali <i>Cartesian Product</i> .	25%
Sub-CPMK 3 (Arsitektur Akses Data C#)	Mahasiswa mampu mengonfigurasi konektivitas ADO.NET modern melalui Microsoft.Data.SqlClient dengan mengimplementasikan standar keamanan Transport Layer Security (TLS/SSL).	25%
Sub-CPMK 4 (Integrasi Antarmuka GUI)	Mahasiswa mampu memvisualisasikan data transaksi gabungan ke dalam kontrol komponen grafis secara terparameterisasi guna mencegah kerentanan keamanan basis data.	25%

2. Metodologi Translasi Model Konseptual ke Model Fisik

Translasi dari *Conceptual Data Model* (CDM/ERD) menuju *Physical Data Model* (PDM) merupakan proses transformasi formal dari entitas independen-teknologi ke skema implementatif yang terikat pada batasan perangkat lunak RDBMS tertentu.

a. Aturan Pemetaan Entitas dan Atribut

Setiap entitas reguler (*strong entity*) pada ERD dikonversi menjadi sebuah tabel fisik (*relation*). Atribut entitas bertransformasi menjadi kolom fisik dengan penentuan tipe data penyimpanan yang proporsional di SQL Server (misalnya, INT, BIGINT, VARCHAR, NVARCHAR, DECIMAL, atau DATETIME). Atribut pengenal (*identifier*) ditetapkan sebagai kunci primer (*Primary Key / PK*) dengan konstrain PRIMARY KEY NOT NULL guna menjamin integritas entitas (*entity integrity*).

b. Aturan Pemetaan Relasi Satu-ke-Satu (1:1)

Relasi 1:1 diimplementasikan dengan menempatkan kunci primer dari salah satu tabel sebagai kunci asing (*Foreign Key / FK*) pada tabel pasangan, yang disertai dengan penambahan konstrain unik (UNIQUE constraint) pada kolom kunci asing tersebut untuk mencegah duplikasi referensi silang.

c. Aturan Pemetaan Relasi Satu-ke-Banyak (1:N)

Relasi 1:N diterjemahkan dengan menanamkan atribut kunci primer dari entitas pada sisi 'satu' (*cardinality of 1*) ke dalam tabel pada sisi 'banyak' (*cardinality of N*) sebagai kolom kunci asing. Kolom ini diproteksi oleh batasan integritas referensial (FOREIGN KEY REFERENCES) yang mengatur aturan penanganan kaskade data (ON UPDATE dan ON DELETE).

d. Aturan Pemetaan Relasi Banyak-ke-Banyak (M:N)

Relasi M:N tidak dapat diimplementasikan secara langsung di dalam struktur tabel relasional tanpa menimbulkan pelanggaran hukum normalisasi pertama. Dekomposisi struktural diwajibkan melalui pembuatan tabel asosiatif (*junction/bridge table*). Kunci primer pada tabel perantara ini dapat

dibentuk melalui kunci gabungan (*composite primary key*) dari kedua kunci asing yang direferensikan, atau menggunakan kunci pengganti (*surrogate key*) dengan penegakan konstrain keunikan gabungan (*composite unique constraint*).

3. Teori Himpunan, Cartesian Product, dan Penggabungan Tabel Multi-Tabel

Operasi pengambilan data multi-tabel dalam SQL berakar pada aljabar relasional (*relational algebra*) dan teori himpunan matematis.

a. Produk Cartesian (Cartesian Product / Cross Join)

Secara matematis, jika relasi R memiliki himpunan baris dengan kardinalitas $|R|$ dan relasi S memiliki himpunan baris dengan kardinalitas $|S|$, maka hasil dari perkalian Cartesian $R \times S$ adalah himpunan pasangan terurut dari setiap baris di R dengan setiap baris di S, yang menghasilkan total baris sebanyak:

$$|R \times S| = |R| \cdot |S|$$

Kondisi *Cartesian Product* terjadi dalam sintaks SQL apabila pengembang menggabungkan dua atau lebih tabel tanpa menyertakan klausa predikat pemadanan (ON atau WHERE). Fenomena ini memicu ledakan kombinatorial (*combinatorial explosion*). Sebagai contoh, penggabungan tabel transaksi berukuran 500.000 baris dengan tabel produk berukuran 10.000 baris tanpa kriteria penggabungan akan menghasilkan 5.000.000.000 baris data sementara di memori, yang menyebabkan saturasi memori RAM, degradasi performa I/O peladen, serta pembekuan aplikasi klien.

b. Prinsip Penggabungan Tabel (Join Mechanism)

Penggabungan tabel secara formal merupakan operasi penyaringan bersyarat (*theta join*) terhadap ruang hasil produk Cartesian. SQL membandingkan nilai atribut penghubung, umumnya antara *Primary Key* pada tabel master dan *Foreign Key* pada tabel transaksional, menggunakan operator relasional persamaan (*equi-join*).



Pemisahan tanggung jawab penulisan kueri sangat dianjurkan:

- Klausa ON: Khusus digunakan untuk mendefinisikan kondisi relasional struktural antar-tabel (*join predicate*).
- Klausa WHERE: Dikhususkan untuk membatasi atau menyaring baris data berdasarkan kriteria logika bisnis (*filter predicate*).

4. Taksonomi Tipe-Tipe JOIN pada Microsoft SQL Server

SQL Server menyediakan sekumpulan operator penggabungan data relasional untuk mengekstraksi irisan maupun perluasan data antar-entitas:

Tabel 2. Operator SQL pada SQL Server

Tipe Operasi Penggabungan	Definisi Logika Himpunan	Perilaku terhadap Data Tak Berpasangan	Contoh Penggunaan Bisnis Nyata
INNER JOIN	Mengembalikan baris data yang memiliki nilai kecocokan yang identik pada kedua tabel yang digabungkan.	Baris dari tabel kiri maupun kanan yang tidak memiliki kecocokan akan dieliminasi dari himpunan hasil.	Menampilkan daftar transaksi yang telah memiliki kepastian data pelanggan terdaftar.
LEFT (OUTER) JOIN	Mengembalikan seluruh baris dari tabel sisi kiri beserta kolom yang cocok dari tabel sisi kanan.	Kolom dari tabel kanan akan diisi dengan nilai NULL apabila tidak ditemukan padanan relasi.	Menampilkan seluruh katalog kategori barang, termasuk kategori yang belum memiliki produk.
RIGHT (OUTER) JOIN	Mengembalikan seluruh baris dari tabel sisi kanan beserta kolom yang cocok dari tabel sisi kiri.	Kolom dari tabel kiri akan diisi dengan nilai NULL apabila rekaman tabel kanan tidak berpasangan.	Menampilkan seluruh data transaksi peminjaman alat, termasuk jika data master mahasiswa telah diarsipkan.
FULL (OUTER) JOIN	Menggabungkan hasil dari Left Join dan Right Join ke dalam satu ruang representasi data.	Menampilkan NULL pada atribut sisi manapun yang tidak memiliki pasangan relasi di tabel seberang.	Audit rekonsiliasi inventaris untuk melacak aset tanpa kategori dan kategori kosong secara serentak.
CROSS JOIN	Menghasilkan perkalian himpunan murni (Cartesian Product) antara kedua tabel tanpa predikat ON.	Seluruh permutasi baris dipertahankan tanpa ada nilai NULL yang diakibatkan oleh ketiadaan referensi.	Pembangkitan kombinasi matriks varian produk (misal: penggabungan semua ukuran dengan semua warna).

5. Rancangan Studi Kasus: Sistem Tata Kelola Inventaris & Sirkulasi Laboratorium

Untuk memvalidasi pemahaman teoritis, studi kasus mengadaptasi skema basis data LabInventoryDB dengan empat tabel relasional yang memuat kasus-kasus batas (*edge cases*), seperti: kategori yang belum memiliki produk dan produk yang belum diasosiasikan ke dalam kategori.

Tabel 3. Struktur data pada dataBase SQL

Nama Tabel Fisik	Nama Atribut Kolom	Tipe Data Penyimpanan	Batasan Nilai / Konstrain	Peran Data
KategoriAlat	IdKategori	INT IDENTITY(1,1)	PRIMARY KEY NOT NULL	Pengenal unik master kategori.
KategoriAlat	NamaKategori	NVARCHAR(50)	NOT NULL	Deskripsi klasifikasi peralatan.
AlatLab	IdAlat	INT IDENTITY(1,1)	PRIMARY KEY NOT NULL	Pengenal unik aset fisik laboratorium.
AlatLab	KodeAlat	VARCHAR(20)	UNIQUE NOT NULL	Kode inventaris laboratorium.
AlatLab	NamaAlat	NVARCHAR(100)	NOT NULL	Nama rinci unit peralatan.
AlatLab	HargaPerolehan	DECIMAL(18,2)	NOT NULL CHECK (HargaPerolehan >= 0)	Nilai valuasi moneter unit aset.
AlatLab	StokTersedia	INT	NOT NULL CHECK (StokTersedia >= 0)	Volume fisik barang yang siap pakai.
AlatLab	IdKategori	INT	NULL FOREIGN KEY	Kunci asing penunjuk ke KategoriAlat.
Peminjam	NIM	VARCHAR(15)	PRIMARY KEY NOT NULL	Identitas unik mahasiswa peminjam.
Peminjam	NamaMahasiswa	NVARCHAR(100)	NOT NULL	Nama lengkap mahasiswa terdaftar.
Peminjam	ProgramStudi	NVARCHAR(50)	NOT NULL	Program studi asal mahasiswa.
PeminjamanHeader	IdPinjam	INT IDENTITY(1,1)	PRIMARY KEY NOT NULL	Nomor transaksi sirkulasi peminjaman.
PeminjamanHeader	NIM	VARCHAR(15)	NOT NULL FOREIGN KEY	Kunci asing penunjuk ke Peminjam.
PeminjamanHeader	TanggalPinjam	DATETIME	NOT NULL DEFAULT GETDATE()	Waktu transaksi disahkan sistem.
PeminjamanHeader	StatusPinjam	VARCHAR(20)	NOT NULL CHECK (StatusPinjam IN ('Aktif', 'Kembali', 'Terlambat'))	Status evaluasi sirkulasi barang.

6. Skrip Implementasi Skema Fisik (Transact-SQL DDL & Sampel Data)

Eksekusikan skrip Transact-SQL berikut di SQL Server Management Studio (SSMS) untuk mengonfigurasi skema fisik dan menyisipkan baris pengujian:

```
-- Membuat Basis Data Baru
CREATE DATABASE LabInventoryJoinDB;
GO

USE LabInventoryJoinDB;
GO

-- 1. Tabel KategoriAlat
CREATE TABLE KategoriAlat (
    IdKategori INT IDENTITY(1,1) NOT NULL,
    NamaKategori NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_KategoriAlat PRIMARY KEY (IdKategori)
);
GO

-- 2. Tabel Master AlatLab (Memperbolehkan NULL pada IdKategori untuk demonstrasi JOIN)
CREATE TABLE AlatLab (
    IdAlat INT IDENTITY(1,1) NOT NULL,
    KodeAlat VARCHAR(20) NOT NULL,
    NamaAlat NVARCHAR(100) NOT NULL,
    HargaPerolehan DECIMAL(18,2) NOT NULL,
    StokTersedia INT NOT NULL,
    IdKategori INT NULL,
    CONSTRAINT PK_AlalatLab PRIMARY KEY (IdAlat),
    CONSTRAINT UQ_AlalatLab_Kode UNIQUE (KodeAlat),
    CONSTRAINT CHK_AlalatLab_Harga CHECK (HargaPerolehan >= 0),
    CONSTRAINT CHK_AlalatLab_Stok CHECK (StokTersedia >= 0),
    CONSTRAINT FK_AlalatLab_Kategori FOREIGN KEY (IdKategori)
        REFERENCES KategoriAlat(IdKategori)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);
GO

-- 3. Tabel Master Peminjam (Mahasiswa)
CREATE TABLE Peminjam (
    NIM VARCHAR(15) NOT NULL,
    NamaMahasiswa NVARCHAR(100) NOT NULL,
    ProgramStudi NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_Peminjam PRIMARY KEY (NIM)
);
GO

-- 4. Tabel Transaksi PeminjamanHeader
CREATE TABLE PeminjamanHeader (
    IdPinjam INT IDENTITY(1,1) NOT NULL,
    NIM VARCHAR(15) NOT NULL,
    TanggalPinjam DATETIME NOT NULL CONSTRAINT DF_Pinjam_Tanggal DEFAULT GETDATE(),
    StatusPinjam VARCHAR(20) NOT NULL,
    CONSTRAINT PK_PeminjamanHeader PRIMARY KEY (IdPinjam),
    CONSTRAINT FK_Peminjaman_Peminjam FOREIGN KEY (NIM)

```

```

REFERENCES Peminjam(NIM)
ON UPDATE CASCADE
ON DELETE NO ACTION,
CONSTRAINT CHK_Peminjaman_Status CHECK (StatusPinjam IN ('Aktif', 'Kembali',
'Terlambat'))
);
GO

-- Injeksi Data Uji dengan Kasus Batas Relasional
INSERT INTO KategoriAlat (NamaKategori) VALUES
('Mikrokontroler'),
('Jaringan Komputer'),
('Alat Ukur Elektronika'),
('Robotika Industri'); -- Kategori ini sengaja tidak memiliki produk (untuk uji
Right/Full Join)

INSERT INTO AlatLab (KodeAlat, NamaAlat, HargaPerolehan, StokTersedia, IdKategori)
VALUES
('MKR-001', 'Arduino Uno R3 Starter Kit', 250000.00, 15, 1),
('MKR-002', 'Raspberry Pi 4 8GB Desktop', 1350000.00, 8, 1),
('NET-001', 'Cisco Router 2901 Integrated', 4500000.00, 3, 2),
('NET-002', 'Managed Switch TP-Link 24-Port', 1200000.00, 5, 2),
('AST-999', 'Solder Station Quick 936A', 450000.00, 10, NULL); -- Produk ini belum
memiliki kategori (untuk uji Left/Full Join)

INSERT INTO Peminjam (NIM, NamaMahasiswa, ProgramStudi) VALUES
('230101001', 'Budi Santoso', 'Teknik Informatika'),
('230101002', 'Dewi Lestari', 'Sistem Informasi'),
('230101003', 'Rian Hidayat', 'Teknik Komputer'); -- Mahasiswa ini belum pernah
meminjam

INSERT INTO PeminjamanHeader (NIM, TanggalPinjam, StatusPinjam) VALUES
('230101001', GETDATE() - 2, 'Aktif'),
('230101002', GETDATE() - 5, 'Kembali');
GO

```

BAGIAN II: PANDUAN PRAKTIKUM IMPLEMENTASI C# VISUAL STUDIO & SQL SERVER

1. Arsitektur Akses Data ADO.NET Modern

Praktikum ini menggunakan pustaka `Microsoft.Data.SqlClient` sebagai penyedia akses data resmi terkini untuk ekosistem .NET. Pustaka `System.Data.SqlClient` terdahulu tidak lagi dikembangkan secara aktif dan tidak mendukung fitur-fitur keamanan terbaru.

Mulai versi 4.0 dan 5.0 dari `Microsoft.Data.SqlClient`, nilai konfigurasi `Encrypt` secara bawaan (*default*) disetel ke `True`. Hal ini mewajibkan seluruh jabat tangan (*handshake*) antara aplikasi C# dan SQL Server terenkripsi dengan sertifikat TLS/SSL.

Pada peladen SQL Server lokal atau instalasi laboratorium yang sering kali memanfaatkan sertifikat terbuat sendiri (*self-signed certificate*), ketiadaan sertifikat otoritas tepercaya (*Trusted Root CA*) pada komputer klien akan memunculkan galat `SqlException`.

Untuk menjamin kelancaran praktikum tanpa menurunkan tingkat keamanan saluran data, kita wajib menyematkan parameter `TrustServerCertificate=True` pada string koneksi. Pengaturan ini menginstruksikan modul klien untuk tetap menggunakan enkripsi TLS pada lapisan transport, namun melewati tahapan validasi rantai sertifikat server.

2. Prosedur Konfigurasi Proyek di Visual Studio 2022

1. Buka Microsoft Visual Studio 2022, pilih menu **Create a new project**.
2. Saring templat dengan memilih bahasa **C#**, platform **Windows**, dan kategori **Desktop**.
3. Pilih templat **Windows Forms App** (berbasis .NET 8.0 LTS modern, bukan .NET Framework klasik), lalu tekan **Next**.
4. Tetapkan nama proyek: `LabJoinViewerApp`, tentukan lokasi folder kerja, lalu pilih target framework: **.NET 8.0 (Long Term Support)**. Klik **Create**.
5. Buka Package Manager Console melalui menu **Tools > NuGet Package Manager > Package Manager Console**, kemudian eksekusikan perintah berikut:

```
Install-Package Microsoft.Data.SqlClient
```

3. Implementasi Kelas Konfigurasi Basis Data

Tambahkan sebuah berkas kelas baru bernama `DbConfig.cs` untuk mengisolasi parameter konektivitas sistem:

```
using System;
using Microsoft.Data.SqlClient;

namespace LabJoinViewerApp
{
    public static class DbConfig
    {
        // Sesuaikan Server dengan instance lokal Anda (misal: localhost atau
        .\SQLEXPRESS)
        private static readonly string ConnString =
```

```

        "Server=localhost;Database=LabInventoryJoinDB;Integrated
Security=True;TrustServerCertificate=True;";

        public static SqlConnection GetConnection()
        {
            return new SqlConnection(ConnString);
        }
    }
}

```

4. Perancangan Antarmuka Grafis Pengguna (FormJoinViewer.cs)

Ubah nama formulir bawaan Form1.cs menjadi FormJoinViewer.cs. Susun komponen visual antarmuka pengguna sesuai spesifikasi teknis berikut:

Nama Komponen	Tipe Kontrol Windows Forms	Konfigurasi Properti Kunci	Tujuan Fungsional
cmbJenisJoin	ComboBox	(Name): cmbJenisJoin, DropDownStyle: DropDownList	Memilih jenis kueri penggabungan yang akan dievaluasi.
btnEksekusi	Button	(Name): btnEksekusi, Text: "Eksekusi Query"	Memicu pengambilan data multi-tabel dari peladen.
dgvHasil	DataGridView	(Name): dgvHasil, ReadOnly: True, AutoSizeColumnsMode: Fill	Menampilkan struktur tabular hasil penggabungan tabel.
lblInformasi	Label	(Name): lblInformasi, Text: "Penjelasan Logika:"	Menampilkan analisis teoritis terkait himpunan kueri aktif.
lblTotalBaris	Label	(Name): lblTotalBaris, Text: "Total Baris: 0"	Menampilkan metrik kuantitas baris hasil kueri.

5. Kode Sumber Logika Program C# (FormJoinViewer.cs)

Tuliskan kode program di bawah ini pada berkas kode pendukung *code-behind* FormJoinViewer.cs:

```

using System;
using System.Data;
using System.Drawing;
using System.Windows.Forms;
using Microsoft.Data.SqlClient;

namespace LabJoinViewerApp
{
    public partial class FormJoinViewer : Form
    {
        public FormJoinViewer()
        {
            InitializeComponent();
        }
    }
}

```

```

private void FormJoinViewer_Load(object sender, EventArgs e)
{
    // Inisialisasi opsi pengujian Join pada ComboBox
    cmbJenisJoin.Items.Clear();
    cmbJenisJoin.Items.Add("1. INNER JOIN (Alat Lab & Kategori Cocok)");
    cmbJenisJoin.Items.Add("2. LEFT OUTER JOIN (Semua Alat Lab & Kategori)");
    cmbJenisJoin.Items.Add("3. RIGHT OUTER JOIN (Semua Kategori & Alat
Terkait)");
    cmbJenisJoin.Items.Add("4. FULL OUTER JOIN (Semua Baris dari Kedua
Tabel)");
    cmbJenisJoin.Items.Add("5. CROSS JOIN (Cartesian Product Alat x
Kategori)");
    cmbJenisJoin.Items.Add("6. MULTI-TABLE INNER JOIN (Transaksi, Peminjam,
Alat)");
    cmbJenisJoin.SelectedIndex = 0;
}

private void btnEksekusi_Click(object sender, EventArgs e)
{
    string sqlQuery = string.Empty;
    string penjelasan = string.Empty;

    // Evaluasi jenis query berdasarkan seleksi ComboBox
    switch (cmbJenisJoin.SelectedIndex)
    {
        case 0: // INNER JOIN
            sqlQuery = @"
                SELECT
                    A.KodeAlat,
                    A>NamaAlat,
                    A.HargaPerolehan,
                    K>NamaKategori
                FROM AlatLab A
                INNER JOIN KategoriAlat K ON A.IdKategori = K.IdKategori;";
            penjelasan = "INNER JOIN: Hanya menampilkan alat yang memiliki
IdKategori valid. " +
                "Perhatikan bahwa 'Solder Station' diabaikan karena
IdKategori bernilai NULL.";
            break;

        case 1: // LEFT OUTER JOIN
            sqlQuery = @"
                SELECT
                    A.KodeAlat,
                    A>NamaAlat,
                    A.HargaPerolehan,
                    ISNULL(K>NamaKategori, '[TIDAK ADA KATEGORI]') AS Kategori
                FROM AlatLab A
                LEFT JOIN KategoriAlat K ON A.IdKategori = K.IdKategori;";
            penjelasan = "LEFT JOIN: Mengambil seluruh rekaman dari tabel kiri
(AlatLab). " +
                "'Solder Station' tetap muncul dengan kolom kategori
terisi nilai NULL.";
            break;

        case 2: // RIGHT OUTER JOIN

```

```

        sqlQuery = @"
            SELECT
                K>NamaKategori,
                ISNULL(A.KodeAlat, '[BELUM ADA ALAT]') AS KodeAlat,
                ISNULL(A>NamaAlat, '-') AS>NamaAlat
            FROM AlatLab A
            RIGHT JOIN KategoriAlat K ON A.IdKategori = K.IdKategori;";
        penjelasan = "RIGHT JOIN: Mengambil seluruh rekaman dari tabel
kanan (KategoriAlat). " +
                    "Kategori 'Robotika Industri' tetap muncul meskipun
belum memiliki unit alat.";
        break;

        case 3: // FULL OUTER JOIN
            sqlQuery = @"
                SELECT
                    A.KodeAlat,
                    A>NamaAlat,
                    K>NamaKategori
                FROM AlatLab A
                FULL OUTER JOIN KategoriAlat K ON A.IdKategori =
K.IdKategori;";
            penjelasan = "FULL JOIN: Menggabungkan seluruh data dari kedua
tabel. " +
                    "Alat tanpa kategori dan kategori tanpa alat sama-sama
dimunculkan dengan NULL.";
            break;

        case 4: // CROSS JOIN (Cartesian Product)
            sqlQuery = @"
                SELECT
                    A.KodeAlat,
                    A>NamaAlat,
                    K>NamaKategori
                FROM AlatLab A
                CROSS JOIN KategoriAlat K;";
            penjelasan = "CROSS JOIN: Menghasilkan perkalian Cartesian murni
(Jumlah Baris Alat * Kategori). " +
                    "Peringatan: Hindari operasi ini pada tabel produksi
tanpa batasan logika!";
            break;

        case 5: // MULTI-TABLE JOIN
            sqlQuery = @"
                SELECT
                    H.IdPinjam,
                    H.TanggalPinjam,
                    P>NamaMahasiswa,
                    P.ProgramStudi,
                    H.StatusPinjam
                FROM PeminjamanHeader H
                INNER JOIN Peminjam P ON H.NIM = P.NIM;";
            penjelasan = "MULTI-TABLE JOIN: Menggabungkan transaksi sirkulasi
dengan entitas induk Peminjam. " +
                    "Mahasiswa yang belum pernah meminjam (Rian Hidayat)
otomatis tersaring keluar.";
            break;

```

```

    }

    EksekusiDataKueri(sqlQuery, penjelasan);
}

private void EksekusiDataKueri(string sql, string deskripsi)
{
    try
    {
        // Penggunaan blok using menjamin pembuangan objek koneksi secara
deterministik
        using (SqlConnection conn = DbConfig.GetConnection())
        {
            using (SqlDataAdapter adapter = new SqlDataAdapter(sql, conn))
            {
                DataTable dataTable = new DataTable();
                adapter.Fill(dataTable);

                // Pengikatan data ke DataGridView
                dgvHasil.DataSource = dataTable;
                lblTotalBaris.Text = $"Total Baris: {dataTable.Rows.Count}";
                lblInformasi.Text = $"Penjelasan Logika: {deskripsi}";

                // Format visual sel jika terdapat nilai string penanda NULL
                FormatTampilanNull();
            }
        }
    }
    catch (SQLException ex)
    {
        MessageBox.Show($"Terjadi kesalahan Transact-SQL: {ex.Message}",
            "Galat Akses Basis Data", MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Kesalahan sistem: {ex.Message}",
            "Galat Umum", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
}

private void FormatTampilanNull()
{
    foreach (DataGridViewRow row in dgvHasil.Rows)
    {
        foreach (DataGridViewCell cell in row.Cells)
        {
            if (cell.Value != null &&
                (cell.Value.ToString().Contains("[TIDAK ADA]") ||
                 cell.Value.ToString().Contains("[BELUM ADA]") ||
                 cell.Value == DBNull.Value))
            {
                cell.Style.BackColor = Color.MistyRose;
                cell.Style.ForeColor = Color.DarkRed;
                cell.Style.Font = new Font(dgvHasil.Font, FontStyle.Italic);
            }
        }
    }
}

```

```

    }
  }
}

```

6. Tugas Praktikum Terbimbing

Mahasiswa diwajibkan menyelesaikan penugasan praktikum berikut:

1. **Analisis Cartesian Product** : Jalankan opsi *CROSS JOIN*. Hitung secara manual berapa jumlah baris di tabel *AlatLab* ($|A|$) dan tabel *KategoriAlat* ($|K|$). Buktikan bahwa jumlah baris yang tampil di *DataGridView* memenuhi formula $|A \times K| = |A| \cdot |K|$.
2. **Kueri Tiga Tabel dengan Pengelompokan** : Buat satu tombol tambahan pada form bernama *btnRekapKategori* yang menjalankan kueri pengelompokan *LEFT JOIN* antara *KategoriAlat* dan *AlatLab* menggunakan fungsi agregasi *COUNT(A.IdAlat)* dan *SUM(ISNULL(A.HargaPerolehan, 0))* untuk menampilkan rekapitulasi total aset per kategori, termasuk kategori yang masih bernilai nol unit.
3. **Analisis Integritas** : Coba lakukan penghapusan salah satu baris data pada tabel *KategoriAlat* yang telah memiliki relasi dengan *AlatLab* langsung melalui SSMS. Amati dan laporkan perilaku konstrain *ON DELETE SET NULL* pada tabel fisik anak.

Selamat Mengerjakan